

Génération d'images à partir de texte

Modèles de diffusion et Flow Matching, de la théorie à l'implémentation

Un cours accompagné d'une implémentation complète en PyTorch

Felix Bos

23 septembre 2026

Table des matières

Avant-propos	3
1 Modèles génératifs : contexte et enjeux	4
1.1 Le problème de la génération	4
1.2 Un bref panorama des approches	4
1.2.1 Modèles à variable latente explicite : les VAE	4
1.2.2 Modèles adversariaux : les GAN	5
1.2.3 Modèles à transport continu : diffusion et flow matching	5
2 Les modèles de diffusion (DDPM)	6
2.1 Vue d'ensemble : deux processus	6
2.2 Le processus direct	6
2.2.1 Formule fermée : bruiteur directement à l'étape t	6
2.3 Le processus inverse	7
2.3.1 Que doit prédire le réseau ?	7
2.4 La fonction de perte	8
2.4.1 Dérivation (aperçu)	8
2.4.2 La loss simplifiée	8
2.5 Algorithmes	8
2.6 Conditionnement par texte et classifier-free guidance	9
2.6.1 Classifier-free guidance	9
3 Flow Matching	10
3.1 Des chemins discrets aux chemins continus	10
3.2 Le champ de vitesse	10
3.3 Génération : résoudre une équation différentielle	11
3.4 Pourquoi (souvent) moins d'étapes suffisent	12
3.5 Diffusion et flow matching : une même famille	13
4 Architecture : le réseau qui apprend à débruiter	14
4.1 Le choix de l'architecture U-Net	14
4.2 Vue d'ensemble du réseau	14
4.3 Encoder le temps : l'embedding sinusoïdal	16
4.4 Le bloc résiduel (ResBlock)	17
4.5 Conditionner sur le texte : la cross-attention	17
4.5.1 Représenter le texte : CLIP	17
4.5.2 Le mécanisme de cross-attention	17
4.5.3 Où placer les blocs de cross-attention ?	18

5 Application : implémentation du projet	19
5.1 Choix de conception	19
5.2 Le jeu de données : CelebA-Dialog	19
5.3 Correspondance théorie $\leftrightarrow$ code	19
5.4 Un exemple de code : la formule fermée du bruitage	19
5.5 Outils d'entraînement	20
5.5.1 CLI unifiée	20
5.5.2 Suivi de l'entraînement	20
5.6 Interface de génération	21
Conclusion et pistes d'approfondissement	22

Avant-propos

Ce document a deux objectifs.

Le premier est pédagogique : présenter, de manière aussi intuitive que rigoureuse, les deux grandes familles de modèles génératifs par transport continu de distribution qui dominent aujourd'hui la génération d'images — les **modèles de diffusion** (DDPM) et le **Flow Matching** — en partant des bases (qu'est-ce qu'un modèle génératif, pourquoi la génération d'images est un problème difficile) jusqu'aux détails d'implémentation (architecture U-Net, conditionnement par texte, classifier-free guidance).

Le second objectif est documentaire : ce cours sert de référence théorique au dépôt de code qui l'accompagne (**text-to-image-generative-models**), une implémentation *from scratch* en PyTorch des deux approches, entraînée sur le jeu de données CelebA-Dialog (visages annotés de légendes en langage naturel), conçue pour tourner entièrement en local sur un Mac Apple Silicon (accélération MPS). Le chapitre 5 fait explicitement le lien entre chaque brique théorique et le fichier de code qui l'implémente.

Intuition

Le fil conducteur de tout ce document tient en une phrase : **un modèle génératif de ce type n'apprend pas à dessiner une image directement** — il apprend une règle de transformation simple (« comment transformer un peu de bruit en quelque chose d'un peu plus net ») et on obtient une image en appliquant cette règle un grand nombre de fois. La diffusion et le flow matching ne diffèrent que par la nature de cette règle et la façon dont on l'apprend.

Comment lire ce document. Les chapitres 1 à 4 constituent le cours proprement dit et sont indépendants du projet : ils se lisent comme une introduction générale aux modèles de diffusion et de flow matching. Le chapitre 5 est spécifique à l'implémentation et suppose les chapitres précédents acquis.

Chapitre 1

Modèles génératifs : contexte et enjeux

1.1 Le problème de la génération

On dispose d'un jeu de données $\{x^{(1)}, \dots, x^{(N)}\}$ — par exemple N photographies de visages — que l'on suppose échantillonnées indépendamment d'une distribution inconnue p_{data} sur un espace de très grande dimension (une image 64×64 en couleur vit dans $\mathbb{R}^{64 \times 64 \times 3} = \mathbb{R}^{12288}$).

Définition 1.1 (Modèle génératif). *Un modèle génératif est une procédure qui, sans connaître p_{data} explicitement, permet de produire de nouveaux échantillons $x \sim p_{\theta}$ où p_{θ} est une distribution apprise à partir des données, censée approcher p_{data} .*

La difficulté centrale n'est pas de mémoriser les données d'entraînement (un modèle qui recopie une image du jeu d'entraînement au hasard « génère » techniquement une image plausible, mais n'a rien appris d'utile) : elle est de capturer la *structure* de p_{data} suffisamment bien pour produire des échantillons **nouveaux** et **plausibles**. Pour des visages, cela veut dire respecter implicitement des milliers de contraintes non écrites (deux yeux symétriques, une peau dont la texture est cohérente avec l'éclairage, des proportions anatomiquement crédibles) sans qu'aucune de ces contraintes n'ait été programmée à la main.

1.2 Un bref panorama des approches

1.2.1 Modèles à variable latente explicite : les VAE

Un *auto-encodeur variationnel* (VAE) apprend conjointement un encodeur $q_{\phi}(z \mid x)$, qui compresse une image x en un vecteur latent z de basse dimension, et un décodeur $p_{\theta}(x \mid z)$, qui reconstruit une image à partir de z . En imposant que z suive approximativement une loi normale $\mathcal{N}(0, I)$, on peut, après entraînement, générer une nouvelle image en tirant $z \sim \mathcal{N}(0, I)$ et en appliquant le décodeur.

Intuition

Le VAE apprend une « carte » compacte de l'espace des images : chaque point de cette carte (le latent z) correspond à une image. Générer, c'est piocher un point au hasard sur la carte.

En pratique, les VAE produisent des images souvent légèrement floues : la loss (une borne inférieure de la vraisemblance, l'ELBO) pénalise moins une image « moyennée » qu'une hallucination nette mais incorrecte, ce qui pousse le modèle vers des reconstructions prudentes.

1.2.2 Modèles adversariaux : les GAN

Un *Generative Adversarial Network* (GAN) oppose deux réseaux : un générateur G qui transforme du bruit en image, et un discriminateur D qui essaie de distinguer les vraies images des images générées. Les deux réseaux s'entraînent simultanément, en compétition — le générateur cherchant à tromper le discriminateur, le discriminateur cherchant à ne pas se laisser tromper.

Les GAN produisent typiquement des images très nettes (pas de flou caractéristique du VAE), mais l'entraînement est notoirement instable (risque d'effondrement de mode, où le générateur cesse de produire une variété d'images et se contente de reproduire quelques exemples qui trompent bien le discriminateur) et ne dispose d'aucune fonction de perte directement interprétable pour suivre la progression.

1.2.3 Modèles à transport continu : diffusion et flow matching

La famille qui nous intéresse dans ce document adopte une stratégie différente : au lieu d'apprendre directement une transformation « bruit $\rightarrow$ image » en une seule étape (comme le générateur d'un GAN), on apprend une transformation **progressive**, décomposée en de nombreux petits pas, chacun faisant passer d'un état légèrement plus bruité à un état légèrement moins bruité (ou l'inverse selon le sens dans lequel on parcourt le processus).

Intuition

Sculpter un bloc de marbre en une statue en un seul geste est presque impossible. Le faire par des milliers de petits coups de burin successifs, chacun corrigeant légèrement la forme précédente, est beaucoup plus faisable — même si aucun coup de burin individuel ne « sait » à quoi ressemblera la statue finale. C'est l'intuition centrale des modèles de diffusion et de flow matching : remplacer un problème de génération difficile (un grand saut) par une séquence de problèmes de *raffinement local* beaucoup plus faciles (de petits pas).

Les deux chapitres suivants détaillent chacune de ces deux approches : les modèles de diffusion (chapitre 2), historiquement les premiers à obtenir des résultats compétitifs avec les GAN sur la génération d'images, puis le flow matching (chapitre 3), une formulation plus récente et plus directe du même principe.

	VAE	GAN	Diffusion / Flow Matching
Entraînement	stable	instable	stable
Netteté des images	moyenne (flou)	élevée	élevée
Diversité des échantillons	bonne	risque d'effondrement	bonne
Coût de génération	1 passe	1 passe	de dizaines à milliers de passes
Loss interprétable	oui (ELBO)	non (jeu à somme nulle)	oui (MSE)

TABLE 1.1 – Comparaison schématique des grandes familles de modèles génératifs d'images.

Le prix à payer pour la stabilité d'entraînement et la qualité des modèles à transport continu est le **coût de génération** : produire une seule image nécessite d'appeler le réseau de neurones de nombreuses fois (typiquement de quelques dizaines à un millier de fois), contre une seule passe pour un GAN ou un VAE. C'est un compromis assumé par toute cette famille de modèles, diffusion comme flow matching.

Chapitre 2

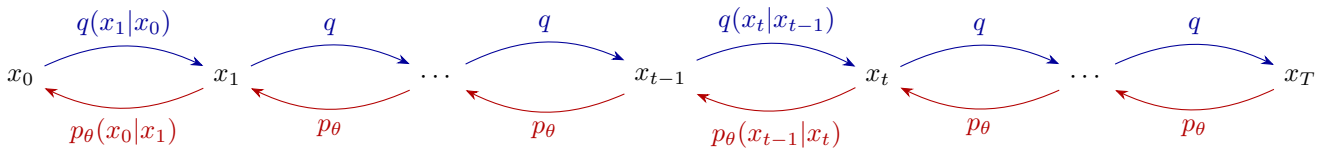
Les modèles de diffusion (DDPM)

Ce chapitre présente les *Denoising Diffusion Probabilistic Models* (DDPM), introduits par Sohl-Dickstein et al. (2015) et popularisés par Ho, Jain et Abbeel (2020) [1], qui sont à l'origine de la vague de modèles text-to-image (DALL-E 2, Imagen, Stable Diffusion).

2.1 Vue d'ensemble : deux processus

Un DDPM est défini par deux processus, tous deux des chaînes de Markov à T étapes ($T \approx 1000$ en pratique) :

- le **processus direct** (*forward process*), noté q , qui bruite progressivement une image x_0 jusqu'à obtenir un bruit gaussien pur x_T . Ce processus est **fixé**, sans paramètre à apprendre ;
- le **processus inverse** (*reverse process*), noté p_θ , qui tente de défaire ce bruitage pas à pas. C'est ce processus, paramétré par un réseau de neurones de poids θ , que l'on entraîne.



Forward q : bruite, fixé, va de gauche à droite **Reverse** p_θ : débruite, appris, va de droite à gauche

2.2 Le processus direct

À chaque étape $t \in \{1, \dots, T\}$, on ajoute une petite quantité de bruit gaussien :

$$q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t I) \quad (2.1)$$

où $(\beta_t)_{t=1}^T$ est un *schedule de bruit* fixé à l'avance, une suite croissante de petites valeurs (typiquement $\beta_1 = 10^{-4}$ à $\beta_T = 2 \times 10^{-2}$, croissance linéaire).

Définition 2.1 (Notations usuelles). On pose $\alpha_t = 1 - \beta_t$ et $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$ (produit cumulé).

2.2.1 Formule fermée : bruite directement à l'étape t

Le résultat clé qui rend le DDPM praticable est qu'on peut bruite x_0 jusqu'à n'importe quelle étape t **sans simuler les $t - 1$ étapes intermédiaires**, grâce à une propriété de composition des lois gaussiennes :

$$q(x_t | x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) I) \quad (2.2)$$

soit, en utilisant le *reparameterization trick* (écrire un tirage gaussien comme une fonction déterministe d'un bruit standard) :

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I) \quad (2.3)$$

Intuition

$\bar{\alpha}_t$ mesure « combien de l'image originale reste » à l'étape t : proche de 1 pour t petit ($x_t \approx x_0$), proche de 0 pour t grand ($x_t \approx \varepsilon$, du bruit pur). La figure 2.1 montre l'évolution de β_t , α_t et $\bar{\alpha}_t$ pour le schedule utilisé dans ce projet.

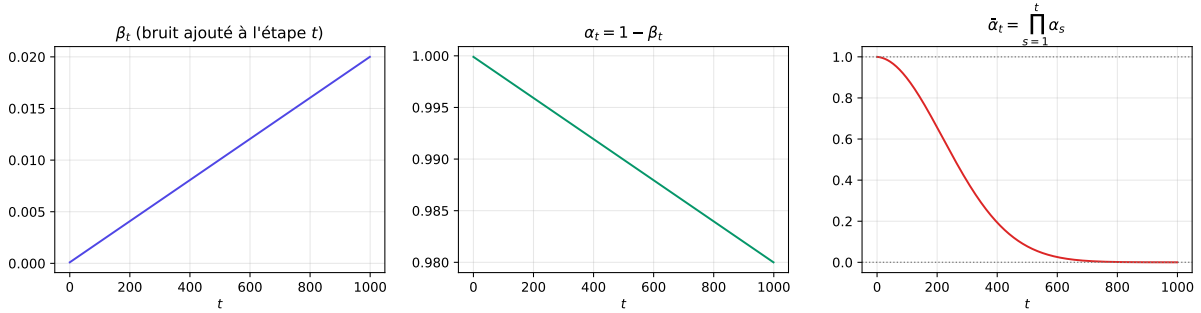


FIGURE 2.1 – Schedule de bruit linéaire ($T = 1000$, $\beta_1 = 10^{-4}$, $\beta_T = 2 \times 10^{-2}$). Notez comment $\bar{\alpha}_t$ (produit cumulé) chute rapidement autour de $t \approx 200-400$ bien que β_t croisse linéairement : c'est un effet cumulatif, pas une propriété de β_t lui-même.

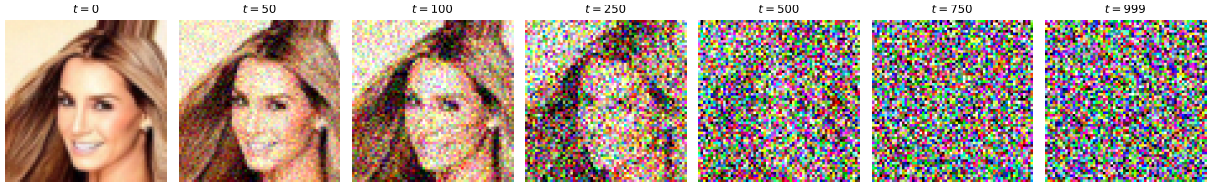


FIGURE 2.2 – Application de la formule (2.3) à une vraie image de visage (CelebA-Dialog), pour plusieurs valeurs de t . L'image reste reconnaissable jusque vers $t \approx 250$, puis se dégrade rapidement en bruit pur — cohérent avec la chute de $\bar{\alpha}_t$ observée figure 2.1.

2.3 Le processus inverse

Défaire le bruitage revient, en théorie, à calculer $q(x_{t-1} | x_t)$ — mais cette distribution dépend de $q(x_{t-1} | x_t, x_0)$, donc de x_0 , que l'on ne connaît justement pas au moment de la génération (c'est précisément ce qu'on cherche à produire). L'idée du DDPM est d'**approcher** cette distribution inconnue par une gaussienne paramétrée :

$$p_\theta(x_{t-1} | x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t)) \quad (2.4)$$

où μ_θ est calculée par un réseau de neurones (le U-Net, chapitre 4), et où Σ_θ est souvent fixée (non apprise) à $\sigma_t^2 I$ avec $\sigma_t^2 = \beta_t$.

2.3.1 Que doit prédire le réseau ?

Une observation cruciale simplifie tout : on peut montrer (Ho et al., 2020) que la moyenne optimale μ_θ s'exprime naturellement en fonction du **bruit** ε qui a été ajouté à x_0 pour produire

x_t (équation (2.3)) :

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \alpha_t}} \varepsilon_\theta(x_t, t) \right) \quad (2.5)$$

où $\varepsilon_\theta(x_t, t)$ est la sortie du réseau de neurones : **au lieu de prédire directement x_{t-1} ou μ_θ , le réseau apprend à prédire le bruit ε** . C'est un choix de paramétrisation, pas une nécessité mathématique, mais Ho et al. montrent empiriquement qu'il conduit à un entraînement bien plus stable et à de meilleurs résultats.

Intuition

Demander au réseau « quelle image propre vois-tu dans ce bruit ? » est une question difficile et mal posée à haut niveau de bruit (beaucoup d'images propres sont compatibles avec un x_t très bruité). Demander « quel bruit a été ajouté ? » est une question plus simple et mieux posée : le réseau n'a qu'à reconnaître le *motif de bruit*, une tâche plus locale et plus uniforme sur toute la plage de t .

Une fois qu'on a μ_θ , générer une étape du reverse process consiste à tirer :

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \alpha_t}} \varepsilon_\theta(x_t, t) \right) + \sigma_t z, \quad z \sim \mathcal{N}(0, I) \text{ si } t > 0, \text{ sinon } z = 0 \quad (2.6)$$

2.4 La fonction de perte

2.4.1 Dérivation (aperçu)

Comme pour un VAE, on entraîne p_θ en maximisant une borne inférieure de la log-vraisemblance (l'ELBO), qui se décompose en une somme de termes de divergence de Kullback-Leibler entre $q(x_{t-1} | x_t, x_0)$ (calculable en forme close) et $p_\theta(x_{t-1} | x_t)$, pour chaque t . Chacun de ces termes, une fois la paramétrisation en ε_θ substituée, se réduit à une erreur quadratique pondérée entre le bruit réel et le bruit prédit. La dérivation complète est disponible dans [1] ; nous n'en retenons ici que le résultat final.

2.4.2 La loss simplifiée

Ho et al. montrent que retirer les poids issus de la dérivation théorique (c'est-à-dire utiliser une simple MSE non pondérée) donne, en pratique, de **meilleurs** résultats visuels :

$$\mathcal{L}_{\text{simple}}(\theta) = \mathbb{E}_{t, x_0, \varepsilon} \left[\left\| \varepsilon - \varepsilon_\theta \left(\sqrt{\alpha_t} x_0 + \sqrt{1 - \alpha_t} \varepsilon, t \right) \right\|^2 \right] \quad (2.7)$$

avec $t \sim \mathcal{U}\{1, \dots, T\}$ et $\varepsilon \sim \mathcal{N}(0, I)$ tirés indépendamment pour chaque exemple d'entraînement. C'est cette loss, remarquablement simple à implémenter (une MSE entre deux tenseurs), qui est utilisée dans ce projet.

2.5 Algorithmes

Algorithme 1 : Entraînement

répéter jusqu'à convergence :

1. tirer $x_0 \sim p_{\text{data}}$ (un batch d'images réelles)
2. tirer $t \sim \mathcal{U}\{1, \dots, T\}$
3. tirer $\varepsilon \sim \mathcal{N}(0, I)$
4. calculer $x_t = \sqrt{\alpha_t} x_0 + \sqrt{1 - \alpha_t} \varepsilon$

5. calculer la loss $\|\varepsilon - \varepsilon_\theta(x_t, t)\|^2$ et rétropropager

Algorithme 2 : Génération (sampling)

1. tirer $x_T \sim \mathcal{N}(0, I)$
2. **pour** $t = T, T-1, \dots, 1$ **faire**
 - (a) calculer $\varepsilon_\theta(x_t, t)$ avec le réseau entraîné
 - (b) calculer x_{t-1} via l'équation (2.6)
3. retourner x_0

Remarque

Contrairement à l'entraînement (un seul t tiré au hasard par exemple, très rapide), la **génération** nécessite T appels séquentiels au réseau — c'est le coût principal des DDPM en pratique, et la motivation première de méthodes comme le flow matching (chapitre 3) ou les schémas d'échantillonnage accélérés (DDIM, etc.), qui visent à réduire ce nombre d'étapes.

2.6 Conditionnement par texte et classifier-free guidance

Jusqu'ici, $\varepsilon_\theta(x_t, t)$ ne dépend que de l'image bruitée et du temps : le modèle est *inconditionnel*, il génère « un visage plausible » sans contrôle sur son contenu. Pour du text-to-image, on conditionne le réseau sur une représentation du texte c (obtenue par un encodeur de texte, voir section 4.5.1) :

$$\varepsilon_\theta(x_t, t, c) \quad (2.8)$$

Le mécanisme précis d'injection de c dans le réseau (cross-attention) est détaillé au chapitre 4.

2.6.1 Classifier-free guidance

Conditionner naïvement ne garantit pas que le modèle *suive fortement* le texte : rien n'empêche le réseau d'apprendre à ignorer partiellement c si les images d'entraînement sont déjà assez prévisibles sans lui. Le *classifier-free guidance* (Ho & Salimans, 2022) [2] est une technique simple et efficace pour renforcer cette influence.

À l'entraînement, on remplace aléatoirement le conditionnement c par un token vide $\emptyset$ avec une probabilité p_{uncond} (typiquement 10 %), de sorte que le même réseau apprenne **à la fois** $\varepsilon_\theta(x_t, t, c)$ (conditionnel) **et** $\varepsilon_\theta(x_t, t, \emptyset)$ (inconditionnel).

À la génération, on combine les deux prédictions avec un facteur d'amplification $w \geq 1$ (le *guidance scale*) :

$$\tilde{\varepsilon}_\theta(x_t, t, c) = \varepsilon_\theta(x_t, t, \emptyset) + w \cdot (\varepsilon_\theta(x_t, t, c) - \varepsilon_\theta(x_t, t, \emptyset)) \quad (2.9)$$

Intuition

Le terme $\varepsilon_\theta(x_t, t, c) - \varepsilon_\theta(x_t, t, \emptyset)$ est la direction « apportée par le texte » : la différence entre ce que le modèle ferait avec et sans conditionnement. L'amplifier par $w > 1$ exagère cette direction, produisant des images qui suivent le prompt plus fidèlement — au prix, si w est trop grand, d'une perte de diversité et d'artefacts visuels (le modèle « sur-corrige »).

Ce mécanisme de dropout du conditionnement à l'entraînement est implémenté dans ce projet (voir chapitre 5); l'application du facteur w au moment du sampling est un point d'extension naturel du code fourni.

Chapitre 3

Flow Matching

Le *Conditional Flow Matching* (Lipman et al., 2023) [3] propose une reformulation du même principe que la diffusion — transformer progressivement du bruit en donnée — mais en des termes plus directs, empruntés à la théorie des équations différentielles ordinaires (ODE) plutôt qu’aux chaînes de Markov gaussiennes.

3.1 Des chemins discrets aux chemins continus

Le DDPM construit implicitement un chemin *stochastique* entre x_0 (donnée) et x_T (bruit), en T pas discrets, chacun ajoutant un peu de bruit aléatoire. Le flow matching part d’une idée plus simple : et si on définissait directement un chemin **déterministe et continu** entre les deux ?

Définition 3.1 (Chemin d’interpolation linéaire). *Pour $t \in [0, 1]$ continu (notez le changement de convention par rapport au DDPM : ici $t = 0$ est la donnée et $t = 1$ est le bruit), on définit :*

$$x_t = (1 - t)x_0 + tx_1, \quad x_0 \sim p_{data}, \quad x_1 \sim \mathcal{N}(0, I) \quad (3.1)$$

C’est un simple segment de droite dans l’espace des images : à $t = 0$ on est exactement sur la donnée, à $t = 1$ exactement sur le bruit, et entre les deux, un mélange linéaire.

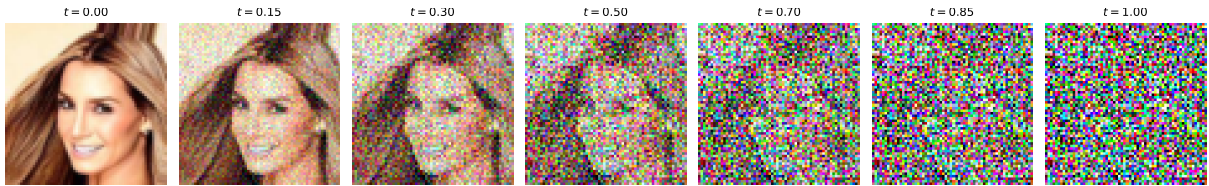


FIGURE 3.1 – Interpolation linéaire (3.1) entre une image réelle et un bruit fixé, pour plusieurs valeurs de t . À comparer avec la figure 2.2 (bruitage DDPM) : la dégradation est ici parfaitement régulière et prévisible, puisque déterministe.

3.2 Le champ de vitesse

Dériver l’équation (3.1) par rapport à t donne la **vélocité** du point le long du chemin :

$$\frac{dx_t}{dt} = x_1 - x_0 \quad (3.2)$$

Remarquez que cette quantité ne dépend pas de t : elle est **constante le long de tout le segment** — c’est la vitesse d’un mouvement rectiligne uniforme entre x_0 et x_1 .

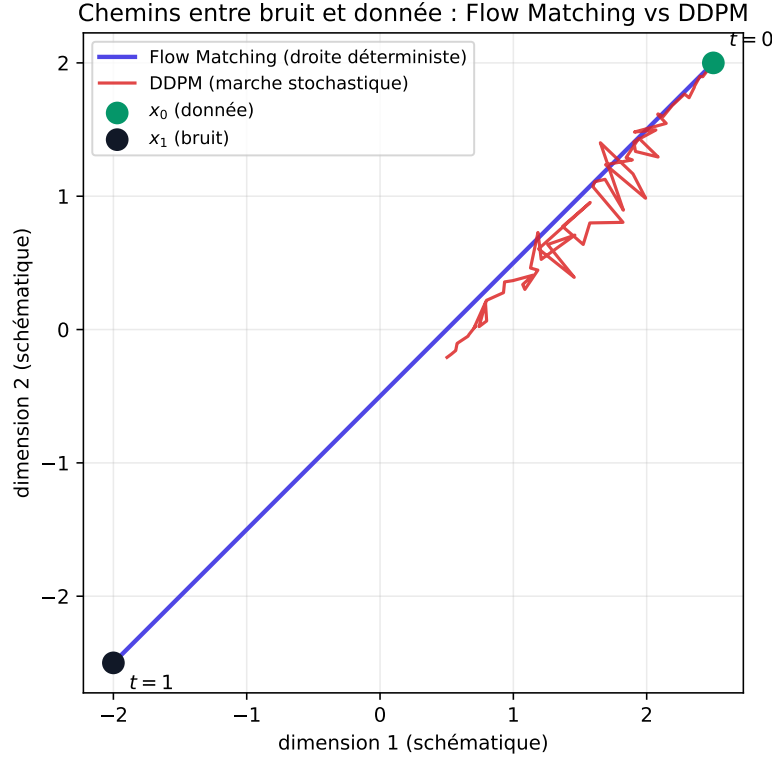


FIGURE 3.2 – Illustration schématique (exemple jouet en 2D, sans rapport avec le vrai espace des images) de la différence entre les deux processus : le flow matching suit une trajectoire rectiligne et déterministe, le DDPM une marche bruitée qui suit le même schedule cumulatif $\bar{\alpha}_t$ mais avec des écarts aléatoires à chaque pas.

Définition 3.2 (Objectif du flow matching). *On entraîne un réseau $v_\theta(x, t)$ à prédire cette vélocité, pour n'importe quel point x sur n'importe quel chemin, à n'importe quel instant t :*

$$\mathcal{L}_{FM}(\theta) = \mathbb{E}_{t \sim \mathcal{U}[0,1], x_0 \sim p_{data}, x_1 \sim \mathcal{N}(0, I)} \left[\|v_\theta((1-t)x_0 + tx_1, t) - (x_1 - x_0)\|^2 \right] \quad (3.3)$$

Intuition

Chaque paire (x_0, x_1) tirée pendant l'entraînement définit *son propre* segment de droite, de sa propre vélocité constante $x_1 - x_0$. Le réseau ne voit jamais un segment entier : à chaque exemple d'entraînement, il ne voit qu'un point x_t tiré sur *un* segment, avec la consigne « la vélocité ici doit valoir $x_1 - x_0$ ». En moyennant sur des millions de tels segments, le réseau apprend, pour toute région de l'espace, une vélocité *moyenne* qui tient compte de tous les segments qui passent à proximité — c'est ce qui permet, à la génération, de suivre un chemin cohérent même en partant d'un point de bruit qui n'a jamais été vu exactement pendant l'entraînement (voir la figure 3.3).

3.3 Génération : résoudre une équation différentielle

Une fois v_θ entraîné, générer une image consiste à résoudre l'équation différentielle ordinaire

$$\frac{dx_t}{dt} = v_\theta(x_t, t) \quad (3.4)$$

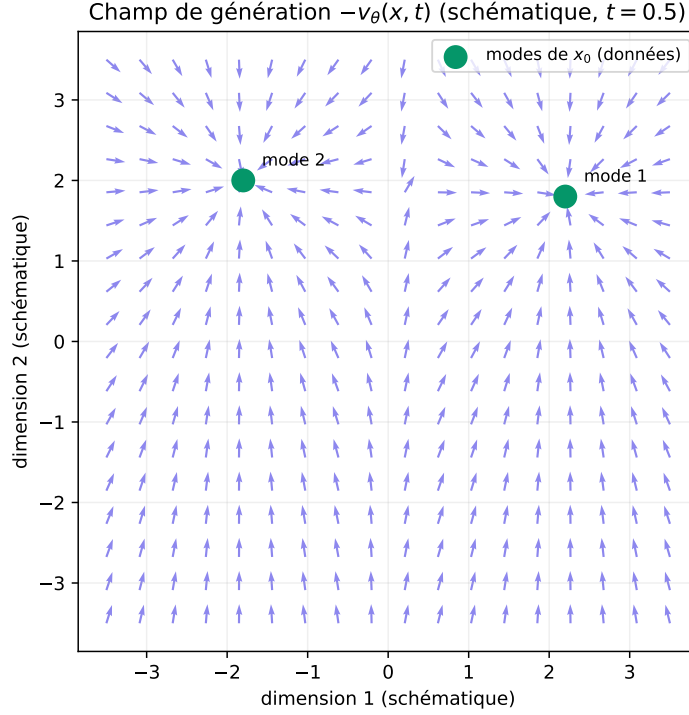


FIGURE 3.3 – Champ de vitesse appris (schématique, exemple 2D jouet à deux modes de données) : à chaque point de l’espace, le réseau prédit une direction. Ici on affiche $-v_\theta$, le sens réellement suivi pendant la génération, qui converge naturellement vers les modes de la distribution de données.

en partant de $x_1 \sim \mathcal{N}(0, I)$ (bruit pur, $t = 1$) et en intégrant **à rebours** jusqu’à $t = 0$. Le schéma numérique le plus simple, l’**Euler explicite**, discrétise cette intégration en N pas de taille $dt = 1/N$:

$$x_{t-dt} = x_t - dt \cdot v_\theta(x_t, t) \quad (3.5)$$

Algorithme : Génération par Flow Matching (Euler explicite)

1. tirer $x_1 \sim \mathcal{N}(0, I)$; poser $t \leftarrow 1$, $dt \leftarrow 1/N$
2. **répéter** N **fois** :
 - (a) calculer $v_\theta(x_t, t)$ avec le réseau entraîné
 - (b) $x_{t-dt} \leftarrow x_t - dt \cdot v_\theta(x_t, t)$; $t \leftarrow t - dt$
3. retourner x_0

Remarque

Contrairement au reverse process du DDPM (équation (2.6)), **aucun bruit aléatoire n’est réinjecté** à chaque étape : la trajectoire complète est entièrement déterminée par le point de départ x_1 et le réseau v_θ . Deux générations avec le même x_1 produisent exactement la même image. C’est une différence qualitative importante avec le DDPM, dont chaque étape du reverse process est stochastique.

3.4 Pourquoi (souvent) moins d’étapes suffisent

Le chemin appris par le flow matching est, par construction, rectiligne au niveau de chaque exemple d’entraînement individuel, et le champ moyen résultant tend à rester relativement « droit ».

Un solveur ODE comme Euler explicite approxime d'autant mieux une trajectoire qu'elle est proche d'une droite : l'erreur de discrétisation d'Euler est en $O(dt)$ par pas, et une trajectoire rectiligne ne requiert presque aucune correction de direction entre deux pas consécutifs. C'est ce qui explique pourquoi, en pratique, le flow matching atteint souvent une qualité comparable avec beaucoup moins de pas de génération ($N \approx 20\text{--}50$) qu'un DDPM classique ($T \approx 250\text{--}1000$), bien que ce ne soit pas une garantie absolue et que cela dépende de la complexité de la distribution de données.

3.5 Diffusion et flow matching : une même famille

Il est utile de noter que DDPM et flow matching, malgré des présentations mathématiques différentes (chaînes de Markov gaussiennes contre ODE continues), appartiennent tous deux à la famille plus large des *modèles à transport continu de distribution*, et peuvent même être reliés formellement (le flow matching avec un chemin gaussien bien choisi retrouve une formulation proche du DDPM en temps continu). Le tableau 3.1 résume les différences pratiques entre les deux implémentations telles qu'utilisées dans ce projet.

	DDPM	Flow Matching
Variable temporelle	$t \in \{0, \dots, T-1\}$ discret	$t \in [0, 1]$ continu
Construction de x_t	schedule $\bar{\alpha}_t$ (non-linéaire)	interpolation linéaire
Sortie du réseau	bruit ε_θ	vitesse v_θ
Cible de la loss	ε (bruit ajouté)	$x_1 - x_0$ (constant sur le chemin)
Génération	stochastique (bruit à chaque pas)	déterministe (ODE)
Nb. d'étapes typique	250 – 1000	20 – 50

TABLE 3.1 – Comparaison pratique des deux approches implémentées dans ce projet.

Chapitre 4

Architecture : le réseau qui apprend à débruiter

Les deux chapitres précédents ont défini *ce que* le réseau doit apprendre à prédire (ε_θ pour le DDPM, v_θ pour le flow matching). Ce chapitre décrit *quel* réseau remplit ce rôle. Un point essentiel structure tout ce chapitre :

Intuition

Le réseau ignore totalement s'il fait de la diffusion ou du flow matching. Il résout un seul et même problème de régression : « étant donné une image x_t dégradée, un temps t , et éventuellement un texte, produis un tenseur de même forme que x_t ». Seule l'interprétation de cette sortie (un bruit ou une vélocité) et la façon dont x_t a été construit diffèrent entre les deux approches. C'est pourquoi une **seule et même architecture** — le U-Net décrit ici — est réutilisée sans aucune modification pour les deux méthodes dans ce projet.

4.1 Le choix de l'architecture U-Net

Le réseau doit accomplir une tâche de *traduction image-à-image* : recevoir une image bruitée et produire une sortie de même résolution spatiale. L'architecture U-Net (Ronneberger et al., 2015, initialement conçue pour la segmentation d'images médicales) est bien adaptée à ce type de tâche :

- une branche **descendante** (*encoder*) réduit progressivement la résolution spatiale tout en augmentant le nombre de canaux, extrayant des caractéristiques de plus en plus abstraites ;
- une branche **montante** (*decoder*), symétrique, restaure progressivement la résolution d'origine ;
- des **connexions résiduelles** (*skip connections*) relient chaque étage de la descente à l'étage de la montée de même résolution, permettant de préserver les détails fins qui seraient sinon perdus dans le goulot d'étranglement central.

4.2 Vue d'ensemble du réseau

La figure 4.1 présente l'architecture complète telle qu'implémentée dans ce projet (configuration par défaut : canaux [64, 128, 256, 256], résolutions $64 \rightarrow 32 \rightarrow 16 \rightarrow 8$).

Trois mécanismes assemblent ce réseau ; nous les détaillons dans le reste du chapitre :

1. l'**injection du temps t** (section 4.3), qui indique au réseau à quel niveau de dégradation il travaille, à chaque étage ;
2. le **bloc résiduel** (*ResBlock*, section 4.4), l'unité de base répétée à chaque étage ;

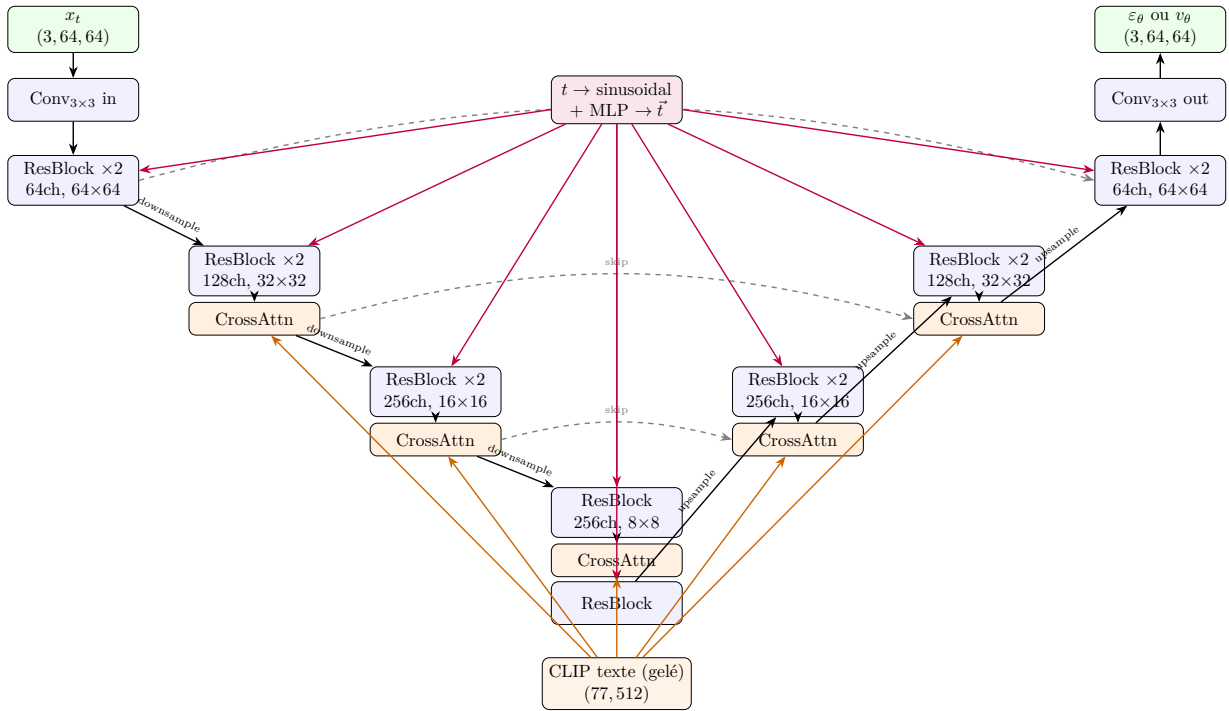


FIGURE 4.1 – Architecture complète du U-Net partagé. Le flux principal (image) traverse le réseau en forme de U (traits noirs pleins). Le temps t (violet) est injecté dans *chaque* bloc résiduel, à toutes les résolutions. Le texte (orange), encodé par CLIP gelé, est injecté par cross-attention aux résolutions 32×32 , 16×16 et au goulot d'étranglement 8×8 — pas à la résolution la plus fine 64×64 , où le coût quadratique de l'attention serait trop élevé pour le bénéfice apporté. Les connexions en pointillés gris sont les skip connections entre étages de même résolution.

3. l'**injection du texte** par cross-attention (section 4.5), qui permet au réseau de conditionner sa prédiction sur une description en langage naturel.

4.3 Encoder le temps : l'embedding sinusoïdal

Le réseau reçoit t (un simple entier, ou un flottant continu pour le flow matching) : un scalaire brut est un signal pauvre et mal exploitable par des couches de convolution. On le transforme d'abord en un vecteur riche, via un *encodage sinusoïdal* — le même principe que le positional encoding des Transformers (Vaswani et al., 2017) :

$$\text{PE}(t, 2i) = \sin\left(\frac{t}{10000^{2i/d}}\right), \quad \text{PE}(t, 2i+1) = \cos\left(\frac{t}{10000^{2i/d}}\right) \quad (4.1)$$

où d est la dimension de l'encodage et $i \in \{0, \dots, d/2 - 1\}$ indexe des fréquences décroissantes. Ce vecteur est ensuite passé dans un petit MLP (**Linear** $\rightarrow$ SiLU $\rightarrow$ **Linear**) pour donner au réseau la capacité d'en tirer une représentation utile à ses propres besoins internes.

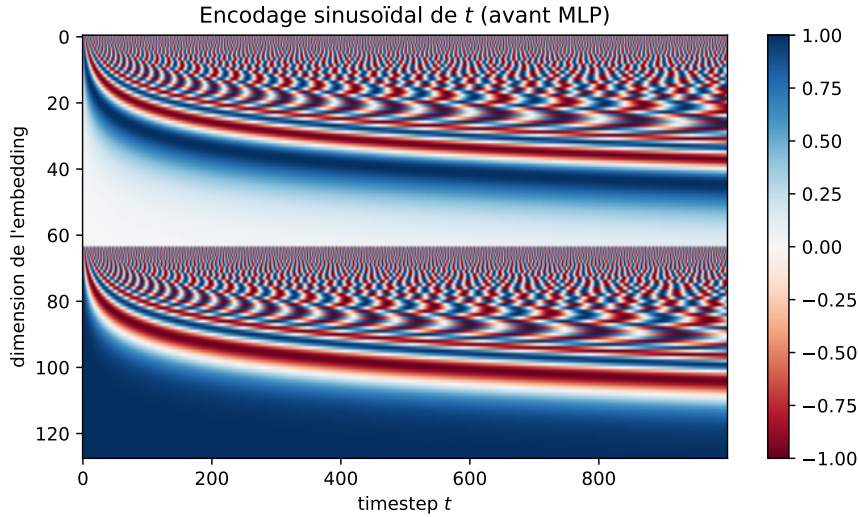


FIGURE 4.2 – Visualisation de l'encodage sinusoïdal brut (avant le MLP) pour $d = 128$ et $t \in [0, 1000]$. Chaque ligne horizontale oscille à sa propre fréquence : les dimensions du haut de chaque moitié (sinus/cosinus) oscillent rapidement (hautes fréquences), celles du bas lentement (basses fréquences). La combinaison de toutes ces fréquences donne, pour chaque valeur de t , une « signature » unique et continue (deux t proches ont des colonnes visuellement proches dans cette figure).

Intuition

C'est l'analogie d'une horloge à plusieurs aiguilles tournant à des vitesses différentes (secondes, minutes, heures) : en lisant la position de toutes les aiguilles simultanément, on identifie un instant précis. Ici, chaque dimension de l'embedding est une « aiguille » tournant à sa fréquence propre en fonction de t .

Le vecteur final, noté $\vec{t} \in \mathbb{R}^{d_{\text{temps}}}$ (dimension 512 dans ce projet), n'est **pas** donné une seule fois en entrée du réseau : il est ré-injecté dans *chaque* ResBlock, à *chaque* étage de résolution (flèches violettes de la figure 4.1).

4.4 Le bloc résiduel (ResBlock)

C'est l'unité de calcul de base, répétée à chaque étage du U-Net. Son rôle est double : extraire des caractéristiques locales de l'image par convolution, et moduler ce calcul en fonction du temps $\vec{t}$.

$$\begin{aligned} h &= \text{Conv}_{3 \times 3}(\text{SiLU}(\text{GroupNorm}(x))) \\ h &= h + \text{Linear}(\vec{t}) \text{ broadcast sur } H, W \\ h &= \text{Conv}_{3 \times 3}(\text{SiLU}(\text{GroupNorm}(h))) \\ \text{sortie} &= h + \text{skip}(x) \end{aligned} \tag{4.2}$$

où $\text{skip}(x) = x$ si le nombre de canaux ne change pas, et $\text{skip}(x) = \text{Conv}_{1 \times 1}(x)$ sinon (pour ajuster la dimension avant l'addition résiduelle).

Intuition

La projection linéaire de $\vec{t}$, additionnée aux feature maps après la première convolution, agit comme un « biais dépendant du contexte » : elle décale toutes les activations d'une quantité qui dépend du niveau de bruit courant, permettant au même jeu de poids de convolution de se comporter différemment selon que l'on débruite une image très bruitée (t grand) ou presque propre (t petit).

La connexion résiduelle ($h + \text{skip}(x)$) suit le principe des ResNets (He et al., 2015) : elle facilite la propagation du gradient à travers un réseau profond (rappelons que le U-Net complet comporte des dizaines de ces blocs empilés), en donnant au réseau la possibilité d'apprendre une simple perturbation de l'entrée plutôt qu'une transformation complète.

4.5 Conditionner sur le texte : la cross-attention

4.5.1 Représenter le texte : CLIP

Comprendre du texte libre (grammaire, synonymes, combinaisons d'attributs) demande d'avoir vu une quantité de langage bien supérieure à ce qu'un jeu de données de quelques centaines de milliers de légendes peut fournir à un encodeur entraîné *from scratch*. Ce projet réutilise donc un encodeur de texte **pré-entraîné et gelé** : le *text encoder* de CLIP (Radford et al., 2021) [4], dont les poids ne sont **jamais mis à jour** pendant l'entraînement — seul le U-Net apprend.

CLIP a été entraîné à aligner des représentations de texte et d'image sur des centaines de millions de paires (image, légende) : ses embeddings capturent donc déjà des notions visuelles pertinentes (« souriant », « lunettes » sont représentés de façon cohérente avec leur contenu visuel), même si CLIP lui-même n'a jamais vu le jeu de données CelebA-Dialog. Une phrase en entrée produit une séquence d'embeddings, un par token :

$$\text{texte} \xrightarrow{\text{CLIP (gelé)}} \text{embed} \in \mathbb{R}^{77 \times 512} \tag{4.3}$$

(77 étant la longueur de séquence maximale standard de CLIP, avec padding pour les phrases plus courtes).

4.5.2 Le mécanisme de cross-attention

Une fois le texte encodé, comment le U-Net l'utilise-t-il ? La réponse est l'**attention croisée** (*cross-attention*), le même mécanisme central des Transformers (Vaswani et al., 2017), mais appliqué entre deux modalités différentes (image et texte) plutôt qu'au sein d'une seule séquence.

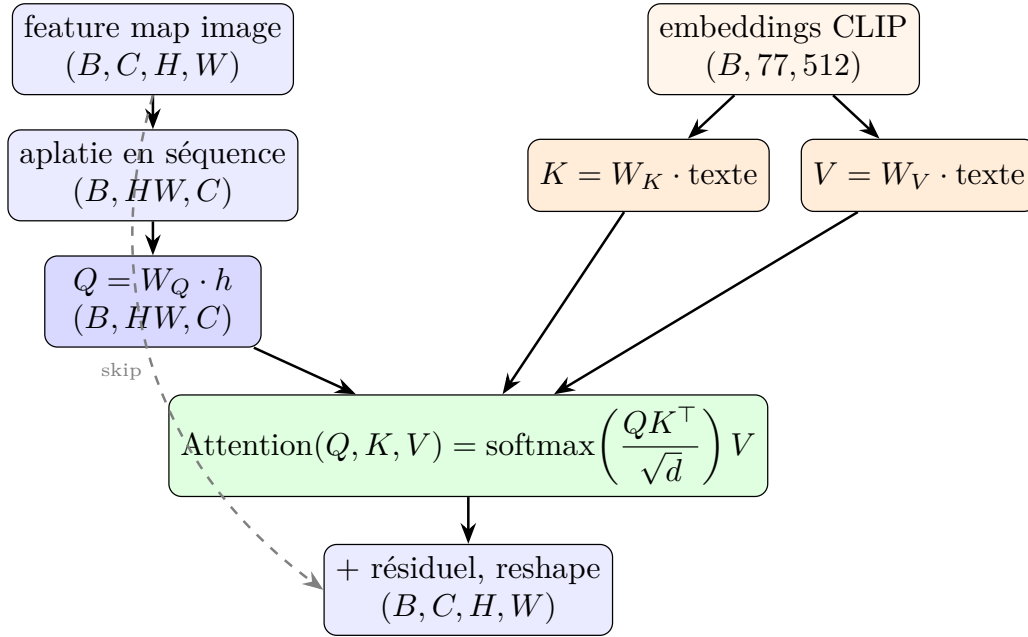


FIGURE 4.3 – Mécanisme de cross-attention texte $\rightarrow$ image utilisé dans chaque bloc **CrossAttentionBlock** du U-Net. Les positions spatiales de la feature map image forment les requêtes (*queries*); les tokens du texte fournissent les clés (*keys*) et valeurs (*values*).

Concrètement, à une résolution donnée, la feature map image de forme (B, C, H, W) est aplatie en une séquence de $H \times W$ « pixels », chacun projeté en une requête Q . Le texte fournit les clés K et valeurs V . L'attention standard est alors calculée :

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V \quad (4.4)$$

Intuition

Chaque position spatiale de l'image « interroge » tous les mots de la phrase et récupère une combinaison pondérée de l'information qu'ils encodent, pondérée par leur pertinence pour cette position précise. Par exemple, les positions spatiales correspondant à la zone de la bouche apprendront, au fil de l'entraînement, à accorder plus de poids aux tokens liés au sourire si la légende en mentionne un.

Comme pour le ResBlock, une connexion résiduelle est appliquée : la sortie de l'attention est **ajoutée** à la feature map d'entrée plutôt que de la remplacer, ce qui permet au réseau d'apprendre progressivement à quel point utiliser l'information textuelle à cet endroit précis du réseau.

4.5.3 Où placer les blocs de cross-attention ?

Le coût de calcul de l'attention standard est quadratique en le nombre de positions ($O((HW)^2)$) : l'appliquer à la résolution la plus fine ($64 \times 64 = 4096$ positions) serait prohibitif. Ce projet suit le choix standard (repris de Stable Diffusion) d'activer la cross-attention uniquement aux résolutions intermédiaires et basses (32×32 , 16×16 , et le goulot d'étranglement 8×8), où le nombre de positions reste raisonnable tout en donnant au réseau suffisamment d'occasions d'intégrer l'information textuelle à différentes échelles de représentation.

Chapitre 5

Application : implémentation du projet

Ce chapitre relie chaque élément théorique des chapitres précédents au code du dépôt (`text-to-image-generative-models`), une implémentation *from scratch* en PyTorch, conçue pour tourner entièrement en local sur un Mac Apple Silicon (accélération MPS).

5.1 Choix de conception

- **Diffusion pixel-space, pas latent-space.** Contrairement à Stable Diffusion, qui diffuse dans l'espace latent compressé d'un VAE, ce projet diffuse directement dans l'espace des pixels. Ce choix simplifie considérablement l'implémentation (pas de VAE à entraîner ou à intégrer) au prix d'une résolution de travail plus faible (64×64), un compromis raisonnable pour un objectif pédagogique réalisable en local.
- **Un seul U-Net, deux approches.** Comme expliqué au chapitre 4, l'architecture U-Net (`src/backend/shared/unet.py`) est strictement identique pour le DDPM et le flow matching : seule la construction de x_t et l'interprétation de la sortie du réseau diffèrent.
- **CLIP gelé.** Le texte est encodé par `openai/clip-vit-base-patch32` (`transformers`), jamais réentraîné.

5.2 Le jeu de données : CelebA-Dialog

Le projet utilise CelebA-Dialog (Jiang et al., 2021) [5], une extension de CelebA qui associe à chacune des 202 599 images de visages une légende en langage naturel générée à partir de cinq attributs à échelle graduée (`Bangs`, `Eyeglasses`, `No_Beard`, `Smiling`, `Young`) : par exemple « *This person is a young smiling woman with long bangs and no eyeglasses.* » Ce jeu de données permet un conditionnement textuel plus riche que de simples labels de classe, sans nécessiter la collecte de vraies légendes humaines pour chaque image, comme le ferait par exemple COCO Captions.

Le module `src/backend/shared/dataset.py` (classe `CelebACaptionDataset`) charge une paire (image, légende), applique le split officiel train/val/test, et exclut automatiquement les quelques images sans annotation disponible.

5.3 Correspondance théorie $\leftrightarrow$ code

5.4 Un exemple de code : la formule fermée du bruitage

L'équation (2.3) se traduit directement en quelques lignes de PyTorch :

Concept théorique	Fichier
Bruitage forward $q(x_t x_0)$, équ. (2.3)	src/backend/diffusion/gaussian_diffusion.py (GaussianDiffusion.q_sample)
Reverse process, équ. (2.6)	GaussianDiffusion.p_sample
Interpolation Flow Matching, équ. (3.1)	src/backend/flow_matching/flow_matching.py (ConditionalFlowMatching.sample_xt)
Encodage sinusoïdal du temps	src/backend/shared/time_embedding.py
ResBlock, CrossAttentionBlock	src/backend/shared/unet_blocks.py
U-Net complet	src/backend/shared/unet.py
Encodeur texte CLIP gelé	src/backend/shared/text_encoder.py
Loss DDPM (MSE sur le bruit) et training loop	src/backend/diffusion/train.py
Loss Flow Matching (MSE sur la vélocité) et training loop	src/backend/flow_matching/train.py
Génération par débruitage itératif (Algo. 2, chap. 2)	src/backend/diffusion/sample.py
Génération par résolution ODE (Euler explicite)	src/backend/flow_matching/sample.py
Classifier-free guidance (dropout à l'entraînement)	argument <code>cfg_dropout_prob</code> des deux <code>train.py</code>

TABLE 5.1 – Correspondance entre les notions théoriques du cours et leur implémentation.

```
def q_sample(self, x_0, t, noise=None):
    if noise is None:
        noise = torch.randn_like(x_0)
    sqrt_alphas_cumprod_t = self._extract(self.sqrt_alphas_cumprod, t, x_0.shape)
    sqrt_one_minus_alphas_cumprod_t = self._extract(
        self.sqrt_one_minus_alphas_cumprod, t, x_0.shape
    )
    x_t = sqrt_alphas_cumprod_t * x_0 + sqrt_one_minus_alphas_cumprod_t * noise
    return x_t, noise
```

La méthode `_extract` gère le fait que chaque image du batch peut avoir un t différent (nécessaire à l'entraînement, où t est tiré aléatoirement pour chaque exemple) : elle sélectionne la bonne valeur précalculée de $\sqrt{\alpha_t}$ pour chaque élément du batch et la redimensionne pour un broadcasting correct sur les dimensions spatiales (H, W).

5.5 Outils d'entraînement

5.5.1 CLI unifiée

L'entraînement, la génération et la comparaison des deux approches passent par une CLI unique (`python -m src.cli`), documentée en détail dans le `README.md` du dépôt :

```
python -m src.cli train diffusion --epochs 5 --batch-size 32
python -m src.cli train flow_matching --epochs 5 --batch-size 32
python -m src.cli sample diffusion --checkpoint checkpoints/diffusion/latest.pt \
    --prompt "a smiling woman with bangs"
python -m src.cli compare --ddpm-checkpoint ... --flow-checkpoint ...
```

5.5.2 Suivi de l'entraînement

Deux mécanismes de suivi sont intégrés :

- **TensorBoard** : chaque `train.py` logge la loss, des grilles d’images générées périodiquement sur un jeu de prompts fixes, et le temps par epoch, dans le dossier `tensorboard/` propre à chaque approche (sous `checkpoints/`).
- **Page de monitoring live** (frontend React, onglet « Suivi entraînement ») : une API FastAPI (`src/frontend/api/main.py`, endpoint `/training-status`) lit ces mêmes logs TensorBoard directement depuis le disque et les expose à une page web qui se rafraîchit automatiquement, sans dépendre de TensorBoard lui-même.

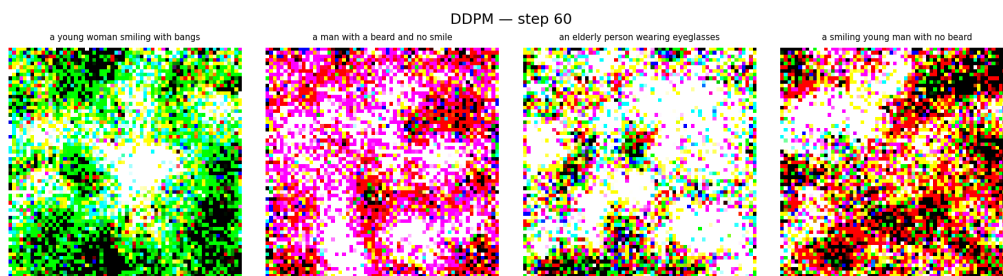


FIGURE 5.1 – Exemple de grille de preview générée automatiquement pendant l’entraînement (ici après seulement 60 pas d’optimisation, à titre d’illustration du mécanisme de suivi — le modèle n’a pas encore convergé et ne produit que du bruit structuré ; un entraînement complet de plusieurs epochs est nécessaire pour obtenir des visages reconnaissables).

5.6 Interface de génération

Un frontend web (React + Vite) et une API FastAPI permettent de générer des images depuis un navigateur, en choisissant l’approche (DDPM ou Flow Matching), le prompt et le nombre d’étapes de sampling. Les modèles sont chargés paresseusement par l’API (au premier appel), ce qui permet de démarrer le serveur sans qu’aucun modèle ne soit encore entraîné.

Conclusion et pistes d’approfondissement

Ce document a présenté les deux grandes approches actuelles de génération d’images par transport continu de distribution — diffusion et flow matching — depuis leurs fondements mathématiques jusqu’à une implémentation complète et fonctionnelle. Les deux méthodes partagent bien plus qu’il n’y paraît au premier abord : la même intuition (remplacer un grand saut génératif par une longue séquence de petits pas), la même architecture de réseau, et ne diffèrent fondamentalement que par la façon dont le chemin entre bruit et donnée est construit (stochastique et non-linéaire pour le DDPM, déterministe et linéaire pour le flow matching).

Pistes d’approfondissement. Plusieurs directions naturelles prolongent ce travail :

- **Diffusion latente.** Remplacer la diffusion pixel-space par une diffusion dans l’espace latent d’un VAE (comme Stable Diffusion), pour viser une résolution supérieure à ressources de calcul égales.
- **Schémas d’échantillonnage accélérés** pour le DDPM (DDIM, solveurs d’ordre supérieur), qui réduisent le nombre d’étapes de génération sans réentraîner le réseau.
- **Fine-tuning de poids pré-entraînés** : charger des poids issus d’un modèle de diffusion existant dans l’architecture U-Net de ce projet, plutôt que d’entraîner entièrement *from scratch*.
- **Évaluation quantitative** (FID, CLIP-score) pour comparer objectivement les deux approches au-delà de l’inspection visuelle.

Bibliographie

- [1] J. Ho, A. Jain, P. Abbeel. *Denoising Diffusion Probabilistic Models*. NeurIPS, 2020.
- [2] J. Ho, T. Salimans. *Classifier-Free Diffusion Guidance*. NeurIPS Workshop on Deep Generative Models, 2022.
- [3] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, M. Le. *Flow Matching for Generative Modeling*. ICLR, 2023.
- [4] A. Radford, J. W. Kim, C. Hallacy, et al. *Learning Transferable Visual Models From Natural Language Supervision (CLIP)*. ICML, 2021.
- [5] Y. Jiang, Z. Huang, X. Pan, C. C. Loy, Z. Liu. *Talk-to-Edit : Fine-Grained Facial Editing via Dialog*. ICCV, 2021.
- [6] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, B. Ommer. *High-Resolution Image Synthesis with Latent Diffusion Models*. CVPR, 2022.
- [7] O. Ronneberger, P. Fischer, T. Brox. *U-Net : Convolutional Networks for Biomedical Image Segmentation*. MICCAI, 2015.
- [8] A. Vaswani, N. Shazeer, N. Parmar, et al. *Attention Is All You Need*. NeurIPS, 2017.